Практическое занятие № 4-5

Паттерны проектирования (Singleton, Factory) + Создание Django-проекта

Цель: Создать Django-проект, реализовать паттерны Singleton и Factory, настроить маршрутизацию.

Краткая теория

Паттерн Singleton (Одиночка)

Паттерн "Одиночка" (Singleton) является одним из паттернов проектирования, который используется для создания класса, имеющего только один экземпляр в системе, и предоставляющего глобальную точку доступа к этому экземпляру.

Это означает, что в рамках приложения может существовать только один объект данного класса, и любой запрос на создание нового экземпляра будет возвращать ссылку на существующий.

Паттерн обеспечивает механизм глобального доступа к единственному экземпляру класса, что упрощает взаимодействие с этим объектом из любой части приложения.

Пример использования:

- Пул подключений к БД (нельзя создавать по подключению на каждый запрос)
- Кэш запросов
- Глобальный логгер

Реализация:

```
class DatabaseConnectionManager:

_instance = None

def __new__(cls):

if cls._instance is None:

cls._instance = super().__new__(cls)

cls._instance.connections = {}
```

```
return cls._instance
```

class Singleton: - Это определение класса Singleton.

_instance = None - Это приватное поле класса, которое будет хранить единственный экземпляр класса. Изначально оно устанавливается в None, что означает, что экземпляр еще не создан.

def **new**(cls): - Это метод **new**, который переопределяется в классе Singleton. Метод **new** вызывается при создании нового объекта. Мы переопределяем его, чтобы контролировать создание экземпляров.

if cls._instance is None: - Это проверка наличия существующего экземпляра класса. Если _instance равно None, это означает, что экземпляр еще не создан, и мы можем создать новый экземпляр.

cls._instance = super(Singleton, cls).**new**(cls) - Здесь мы создаем новый экземпляр класса, если он еще не существует. Мы используем функцию super() для вызова базовой реализации метода **new**, которая фактически создает новый экземпляр класса.

return cls._instance - Мы возвращаем существующий или только что созданный экземпляр класса. Это дает уверенность в том, что всегда будет использоваться только один экземпляр класса.

Паттерн Factory (Фабрика)

Паттерн «Фабрика» создаёт объекты, не указывая их точный класс.

Вместо прямого вызова конструкторов используется фабричный метод, который решает, какой класс создать.

Пример использования:

- Создание разных типов серверов СУБД (PostgreSQL, MySQL, Oracle)
- Создание разных типов уведомлений (Email, SMS, мессенджеры)
- Создание разных типов отчётов (PDF, Excel, CSV)

Реализация:

```
class ServerFactory:

_registry = {

"postgresql": PostgreSQLServer,
```

```
"mysql": MySQLServer,

"oracle": OracleServer,

}

@classmethod

def create_server(cls, server_type, name, ip):

if server_type not in cls._registry:

raise ValueError(f"Неизвестный тип: {server_type}")

return cls._registry[server_type](name, ip)
```

Практическая часть. Создание «тренировочного» Django-проекта

ВАЖНО: правильная структура

Django создаёт **вложенную** структуру. Чтобы избежать путаницы, используйте имя `config` для папки настроек.

Что мы делаем: создаем Django-приложение (app) в составе Django-проекта. **Модуль** большой системы, который отвечает за одну конкретную задачу - учёт серверов СУБД.

В реальном enterprise-проекте наше приложение servers было бы одним из десятков модулей.

То, что мы сделаем, практически прототип реального модуля.

Подобные модули существуют, например в pgAdmin (графический клиент для работы с PostgreSQL):

- модуль servers — список серверов
- модуль databases — список БД
- модуль backups — бэкапы
- модуль users — пользователи

Шаги

Откройте терминал в PyCharm и выполните команды по одной:

```
# 1. Переходим в папку проектов

cd ваш путь\PycharmProjects
```

```
# 2. Создаём папку для проекта

mkdir db_project

cd db_project
```

```
# 3. Создаём виртуальное окружение

python -m venv venv
```

```
# 4. Активируем его (если Windows PowerShell)

.\venv\Scripts\Activate.ps1

# 5. Устанавливаем Django

pip install django

# 6. Создаём проект с ТОЧКОЙ в конце!

# Точка означает «создать в текущей папке, без вложенности»

django-admin startproject config .

# 7. Создаём приложение

python manage.py startapp servers
```

Правильная структура проекта (обязательно сверить, если отличается — скорректировать, либо создать заново)

```
db_project/
├── venv/                    ← виртуальное окружение
├── manage.py                ← ОДИН manage.py в корне
├── config/                  ← настройки проекта
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
└── servers/                 ← ваше приложение
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── views.py
    └── tests.py
```

Настройка проекта

1. Зарегистрируйте приложение в `config/settings.py`:

Найдите секцию `INSTALLED_APPS` и добавьте `'servers'`:

```
INSTALLED_APPS = [

'django.contrib.admin',

'django.contrib.auth',

'django.contrib.contenttypes',

'django.contrib.sessions',

'django.contrib.messages',

'django.contrib.staticfiles',

'servers', # ← ЭТУ СТРОКУ!

]
```

2. Создайте файл `servers/urls.py` (Django его **не** создаёт автоматически, нужно создать самим):

```
from django.urls import path

from . import views


urlpatterns = [

path('', views.index, name='index'),

]
```

3. Подключите urls приложения в `config/urls.py`**:**

```
from django.contrib import admin

from django.urls import path, include


urlpatterns = [

path('admin/', admin.site.urls),

path('servers/', include('servers.urls')), # ← НОВАЯ СТРОКА

]
```

4. Создайте простую view в `servers/views.py`**:**

```
from django.http import HttpResponse


def index(request):

return HttpResponse("Система администрирования БД работает!")
```

Проверка

```
python manage.py runserver
```

Откройте `http://127.0.0.1:8000/servers/` — должна появиться строка "Система администрирования БД работает!"

При проблемах с запуском еще раз проверьте структуру проекта

Реализация паттернов

Задание 1: Singleton

Создайте файл `servers/singleton.py` :

```
class DatabaseConnectionManager:

"""Singleton: управляет подключениями к разным СУБД."""

_instance = None

def __new__(cls):

if cls._instance is None:

cls._instance = super().__new__(cls)

cls._instance.connections = {}

return cls._instance

def get_connection(self, db_type: str) -> str:

if db_type not in self.connections:

self.connections[db_type] = f"Connection to {db_type}"

return self.connections[db_type]
```

```
def get_all_connections(self) -> dict:

return self.connections
```

Проверим работу: создаем `servers/test_singleton.py` и убедитесь, что `manager1 is manager2` возвращает нам `True`.

Задание 2: Factory

Создайте в `servers/models.py` базовый класс и наследников:

```
class DatabaseServer:

def __init__(self, name: str, ip_address: str):

self.name = name

self.ip_address = ip_address

def get_info(self) -> str:

raise NotImplementedError()


class PostgreSQLServer(DatabaseServer):

def get_info(self) -> str:

return f"PostgreSQL: {self.name} ({self.ip_address}:5432)"


class MySQLServer(DatabaseServer):

def get_info(self) -> str:

return f"MySQL: {self.name} ({self.ip_address}:3306)"


class ServerFactory:

_registry = {

"postgresql": PostgreSQLServer,
```

```
"mysql": MySQLServer,

}

@classmethod

def create_server(cls, server_type: str, name: str, ip: str):

if server_type not in cls._registry:

raise ValueError(f"Неизвестный тип: {server_type}")

return cls._registry[server_type](name, ip)
```

Протестируем: сразу создаем `servers/test_factory.py` и убеждаемся, что фабрика создаёт объекты разных типов.

Задание 3: Интеграция с Django

Обновите файл `servers/views.py`. После обновления запустите проект и посмотрите на результат.

```
from django.http import HttpResponse

from servers.singleton import DatabaseConnectionManager

from servers.models import ServerFactory



def index(request):

manager = DatabaseConnectionManager()

manager.get_connection("postgresql")

pg_server = ServerFactory.create_server("postgresql", "prod-db",
"192.168.1.10")

html = f"""

<h1>Система учёта серверов СУБД</h1>

<p>Singleton ID: {id(manager)}</p>

<p>Подключения: {manager.get_all_connections()}</p>
```

```
    <p>Сервер: {pg_server.get_info()}</p>
    """
    return HttpResponse(html)
```

Индивидуальное задание (Оценка)

Назначенный вариант укажите в работе.
Реализуйте свою предметную область, используя Singleton и Factory, как в примере.

Блок вариантов 1: Общая тема: Управление инфраструктурой БД

1. Система учёта серверов СУБД (НУЛЕВОЙ ВАРИАНТ **пример из занятия**)

- Singleton: `DatabaseConnectionManager`
- Factory: `ServerFactory` (PostgreSQL, MySQL, Oracle)

2. Журнал изменений схем БД

- Singleton: `SchemaChangeLogger` (единый логгер изменений)
- Factory: `ChangeFactory` (ALTER, CREATE, DROP, RENAME)

3. Система управления пользователями БД

- Singleton: `UserSessionManager`
- Factory: `UserFactory` (Admin, DBA, Developer, Readonly)

4. Мониторинг репликации БД

- Singleton: `ReplicationMonitor`
- Factory: `ReplicationStatusFactory` (Master, Slave, Delayed)

5. Учёт хранимых процедур

- Singleton: `ProcedureRegistry`
- Factory: `ProcedureFactory` (Function, Trigger, View)

Блок вариантов 2: Общая тема: Безопасность и аудит

6. Система аудита действий в БД

- Singleton: `AuditLogger`
- Factory: `AuditEventFactory` (SELECT, INSERT, UPDATE, DELETE)

7. Учёт инцидентов ИБ с БД

- Singleton: `IncidentManager`
- Factory: `IncidentFactory` (DDoS, Phishing, DataLeak)

8. Система управления доступом к БД (DAM)

- Singleton: `AccessManager`
- Factory: `AccessRequestFactory` (Read, Write, Admin)

9. Журнал подозрительных запросов

- Singleton: `SuspiciousQueryLogger`
- Factory: `QueryTypeFactory` (Slow, NoIndex, Injection)

10. Система ротации секретов

- Singleton: `SecretsRotator`
- Factory: `SecretFactory` (Password, APIKey, Certificate)

Блок вариантов 3: Общая тема: Производительность

11. Мониторинг медленных запросов

- Singleton: `SlowQueryMonitor`
- Factory: `QueryReportFactory` (Daily, Weekly, Monthly)

12. Учёт индексов

- Singleton: `IndexRegistry`
- Factory: `IndexFactory` (BTree, Hash, GIN, GiST)

13. Система управления тестовыми данными

- Singleton: `TestDataManager`
- Factory: `DataGeneratorFactory` (Users, Orders, Products)

14. Мониторинг свободного места

- Singleton: `StorageMonitor`
- Factory: `AlertFactory` (Warning, Critical, Emergency)

15. Система алертов по метрикам

- Singleton: `AlertManager`
- Factory: `NotificationFactory` (Email, SMS, VK)

Требования к индивидуальной работе

Обязательные элементы

Singleton:

- Реализован через `__new__`
- Продемонстрировано, что создаётся один экземпляр (`id(obj1) == id(obj2)`)
- Используется в view Django

Factory:

- Базовый класс с методом (например, `get_info()` , `execute()` , `generate()`)
- Минимум 2 класса-наследника
- Фабрика с методом `create()`
- Используется в view Django

Маршрутизация:

- Минимум 2 URL в приложении
- Главная страница показывает работу Singleton и Factory

Тесты:

- `test_singleton.py` — проверка единственности экземпляра
- `test_factory.py` — проверка создания разных типов

Защита работы

Подготовиться к вопросам

1. **"Почему вы использовали `__new__` , а не `__init__` для Singleton?"**
2. **"Что будет, если добавить ещё один тип в вашу Factory?"**
3. **"Где в реальном администрировании БД это пригодится?"**
4. **"Покажите, что Singleton действительно создаёт один экземпляр."**
5. **"Как изменить URL-адреса в вашем приложении?"**

Демонстрация работы

- Добавить новый тип в Factory прямо на занятии
- Изменить URL-маршрут
- Показать, что происходит при ошибке (например, неверный тип сервера)

Перед сдачей работы проверьте:

Структура

- [ ] `manage.py` — один, в корне
- [ ] Папка `config/` с `settings.py`, `urls.py`
- [ ] Папка `servers/` с `urls.py`, `views.py`, `singleton.py`, `models.py`
- [ ] Приложение `servers` зарегистрировано в `INSTALLED_APPS`

Функциональность

- [ ] `python manage.py runserver` запускается без ошибок
- [ ] Главная страница открывается
- [ ] Singleton создаёт один экземпляр
- [ ] Factory создаёт разные типы объектов
- [ ] Тесты проходят

Код

- [ ] Есть комментарии
- [ ] Понятные имена переменных
- [ ] Обработка ошибок (`try/except`)